

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

William Fedus*

LIAMFEDUS@GOOGLE.COM

Barret Zoph*

BARRETZOPH@GOOGLE.COM

Noam Shazeer

NOAM@GOOGLE.COM

Google, Mountain View, CA 94043, USA

Editor: Alexander Clark

Abstract

In deep learning, models typically reuse the same parameters for all inputs. Mixture of Experts (MoE) models defy this and instead select *different* parameters for each incoming example. The result is a sparsely-activated model—with an outrageous number of parameters—but a constant computational cost. However, despite several notable successes of MoE, widespread adoption has been hindered by complexity, communication costs, and training instability. We address these with the introduction of the Switch Transformer. We simplify the MoE routing algorithm and design intuitive improved models with reduced communication and computational costs. Our proposed training techniques mitigate the instabilities, and we show large sparse models may be trained, for the first time, with lower precision (bfloat16) formats. We design models based off T5-Base and T5-Large (Raffel et al., 2019) to obtain up to 7x increases in pre-training speed with the same computational resources. These improvements extend into multilingual settings where we measure gains over the mT5-Base version across all 101 languages. Finally, we advance the current scale of language models by pre-training up to trillion parameter models on the “Colossal Clean Crawled Corpus”, and achieve a 4x speedup over the T5-XXL model.¹²

Keywords: mixture-of-experts, natural language processing, sparsity, large-scale machine learning, distributed computing

*. Equal contribution.

1. JAX code for Switch Transformer and all model checkpoints are available at <https://github.com/google-research/t5x>
2. Tensorflow code for Switch Transformer is available at https://github.com/tensorflow/mesh/blob/master/mesh_tensorflow/transformer/moe.py

Contents

1	Introduction	3
2	Switch Transformer	4
2.1	Simplifying Sparse Routing	5
2.2	Efficient Sparse Routing	6
2.3	Putting It All Together: The Switch Transformer	8
2.4	Improved Training and Fine-Tuning Techniques	8
3	Scaling Properties	11
3.1	Scaling Results on a Step-Basis	12
3.2	Scaling Results on a Time-Basis	13
3.3	Scaling Versus a Larger Dense Model	13
4	Downstream Results	14
4.1	Fine-Tuning	14
4.2	Distillation	16
4.3	Multilingual Learning	17
5	Designing Models with Data, Model, and Expert-Parallelism	18
5.1	Data Parallelism	20
5.2	Model Parallelism	20
5.3	Model and Data Parallelism	21
5.4	Expert and Data Parallelism	22
5.5	Expert, Model and Data Parallelism	22
5.6	Towards Trillion Parameter Models	22
6	Related Work	24
7	Discussion	25
8	Future Work	26
9	Conclusion	27
A	Switch for Attention	27
B	Preventing Token Dropping with <i>No-Token-Left-Behind</i>	29
C	Encouraging Exploration Across Experts	29
D	Switch Transformers in Lower Compute Regimes	29
E	Relation of Upstream to Downstream Model Performance	32
F	Pseudo Code for Switch Transformers	33

1. 引言

大规模训练已成为通往灵活而强大的神经语言模型的有效路径（Radford 等，2018 年；Kaplan 等，2020 年；Brown 等，2020 年）。简单架构——在充足的计算预算、数据集规模和参数数量的支撑下——优于更复杂的算法（Sutton, 2019 年）。Radford 等（2018 年）；Raffel 等（2019 年）；Brown 等（2020 年）所采用的一种方法是扩大稠密激活的 Transformer（Vaswani 等，2017 年）的模型规模。虽然有效，但其计算开销也极其巨大（Strubell 等，2019 年）。受模型规模成功的启发，但为了获得更高的计算效率，我们提出了一种稀疏激活的专家模型：Switch Transformer。在我们的设置中，稀疏性来自于对每个输入样本仅激活神经网络权重的子集。

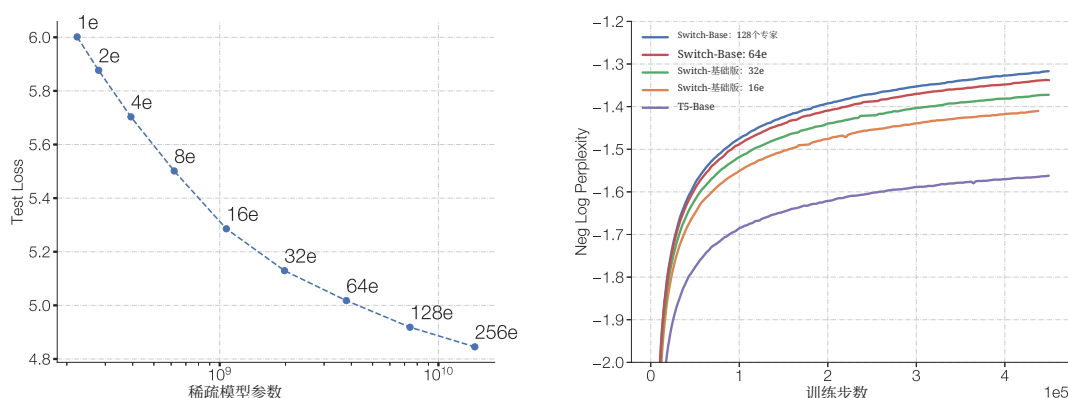


图 1: Switch Transformer 的扩展性与样本效率。左图：随稀疏性增加（更多专家）的 Switch Transformer 的扩展性特性。右图：在相同计算预算下，将 Switch Transformer 与 T5（Raffel 等，2019 年）模型比较的负对数困惑度。

稀疏训练是一个活跃的研究与工程领域（Gray 等，2017 年；Gale 等，2020 年），但截至目前，机器学习库和硬件加速器仍主要面向稠密矩阵乘法。为了获得高效的稀疏算法，我们从专家混合（MoE）范式出发（Jacobs 等，1991 年；Jordan 和 Jacobs, 1994 年；Shazeer 等，2017 年），并对其加以简化，以带来训练稳定性和计算优势。MoE 模型在机器翻译方面取得了显著成功（Shazeer 等，2017 年、2018 年；Lepikhin 等，2020 年），然而，其广泛采用受到复杂性、通信开销和训练不稳定性的阻碍。

我们解决了这些问题，并进一步超越翻译，发现这类算法在自然语言中具有广泛价值。我们在多样的自然语言任务上以及自然语言处理的三个阶段——预训练、微调和多任务训练——测得了优越的扩展性。尽管本工作侧重于规模，我们也表明，Switch Transformer 架构不仅在超级计算机领域表现出色，而且

即使只配备少量计算核心也同样有益。此外，我们的大型稀疏模型可以被蒸馏（Hinton 等，2015）成小型稠密版本，同时保留稀疏模型质量增益的30%。我们的贡献如下：

- Switch Transformer 架构，对专家混合进行了简化并带来改进。
- 扩展性特性以及与经过强力调优的 T5 模型（Raffel 等，2019 年）的基准对比，我们测得 7 倍+预训练加速，同时仍使用相同的每个标记的FLOPs。我们进一步表明，即使在计算资源受限、仅使用两个专家的情况下，这些改进也依然成立。
- 成功地将稀疏预训练并专门微调的模型蒸馏为小型稠密模型。我们将模型规模最多减少 99%，同时保留大型稀疏教师模型 30% 的质量提升。
- 改进的预训练和微调技术：(1) 选择性精度训练，使得可以在更低的 bfloat16 精度下进行训练(2) 一种初始化方案，允许扩展到更大的专家数量，以及(3) 增强的专家正则化，改善稀疏模型微调和多任务训练。
- 对多语言数据上的预训练收益进行测量，我们发现所有 101种语言均有普遍改进，其中有 91% 的语言相较于 mT5 基线（Xue 等，2020）获得了 4 倍+ 加速。
- 通过高效结合数据、模型和专家并行，实现神经语言模型的规模提升，从而构建参数数量高达万亿的模型。这些模型将高度调优的 T5-XXL 基线的预训练速度提升了 4 倍。

2. Switch Transformer

Switch Transformer 的指导性设计原则，是以一种简单且计算高效的方式最大化 Transformer 模型（Vaswani 等，2017 年）的参数数量。Kaplan 等（2020）对扩展性的益处进行了全面研究，发现模型规模、数据集规模与计算预算遵循幂律缩放。重要的是，该工作主张在相对较少的数据上训练大规模模型，这是在计算上最优的做法。

鉴于这些结果，我们研究第四个扩展维度：在保持每个样本的浮点运算量（FLOPs）不变的情况下，增加参数数量。我们的假设是，参数数量独立于所执行的总计算，是另一个需要单独扩展的重要维度。我们通过设计一种稀疏激活模型来实现这一点，该模型能够高效利用为稠密矩阵乘法而设计的硬件，如 GPU 和 TPU。我们的工作主要聚焦于 TPU 架构，但这类模型也可以在 GPU 集群上进行类似的训练。在我们的分布式训练设置中，稀疏激活层会将唯一的权重拆分到不同的设备上。因此，随着设备数量的增加，模型的权重也随之增长，同时在每个设备上仍保持可控的内存与计算开销。

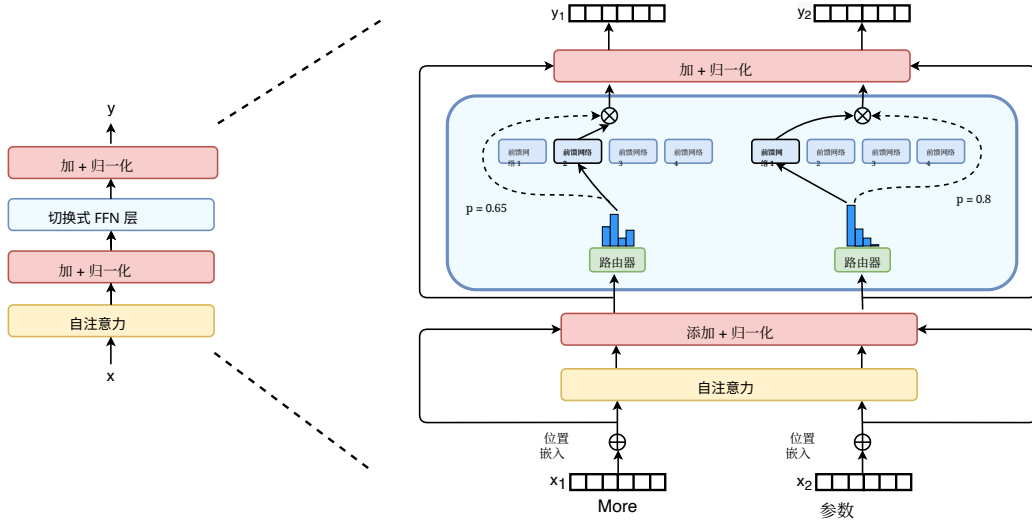


图2: Switch Transformer 编码器块示意图。我们将 Transformer 中的稠密前馈网络 (FFN) 层替换为稀疏的 Switch FFN 层 (浅蓝色)。该层对序列中的标记独立地进行操作。我们示意两个标记 (下文中的 x_1 =“更多”和 x_2 =“参数”) 在四个 FFN 专家之间已路由 (实线), 其中路由器独立地为每个标记进行路由。该切换式 FFN 层返回所选 FFN 的输出与路由器门控值相乘的结果 (虚线)。

2.1 简化稀疏路由

专家混合路由。 Shazeer 等 (2017) 提出了一种自然语言 专家混合 (MoE) 层, 其将一个标记表示 x 作为输入, 然后将其路由到从集合 $\{E_i(x)\}_{i=1}^N$ 的 N 位专家中选出的、最优的前- k 位专家。路由器变量 W_r 生成 logits $h(x) = W_r \cdot x$, 并通过该层可用的 N 位专家上的 softmax 分布进行归一化。专家 i 的门值给出为,

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}}. \quad (1)$$

为路由标记 x , 选择 top- k 门控值。如果 $\mathcal{T}$ 是所选 top- k 索引的集合, 则该层的输出计算是各专家对该标记的计算按门控值进行的线性加权组合,

$$y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x). \quad (2)$$

Switch 路由: 重新思考专家混合。 Shazeer 等 (2017) 推测, 为了使路由函数具有非平凡的梯度, 必须路由到 $k > 1$ 位专家。作者直觉认为, 如果无法比较至少两个专家, 学习路由将无法奏效。Ramachandran 和 Le (2018) 更进一步地

研究了 $\text{top-}k$ 决策，并发现，在模型的较低层中更高的 k 值对于具有许多路由层的模型很重要。与这些看法相反，我们改用一种简化策略，只将路由发送到单个专家。我们表明，这种简化既保持了模型质量，又减少了路由计算，并且表现更好。这种 $k = 1$ 路由策略后来被称为 **Switch** 层。请注意，对于 MoE 和 Switch 路由，公式 2 中的门控值 $p_i(x)$ 使路由器可微。

Switch 层的益处有三方面：(1) 由于我们只将一个标记路由到单个专家，路由器计算减少。(2) 由于每个标记只被路由到单个专家，每位专家的批大小（专家容量）至少可以减半。³(3) 路由实现被简化，通信开销也随之降低。图 3 展示了在不同专家容量系数下的路由示例。

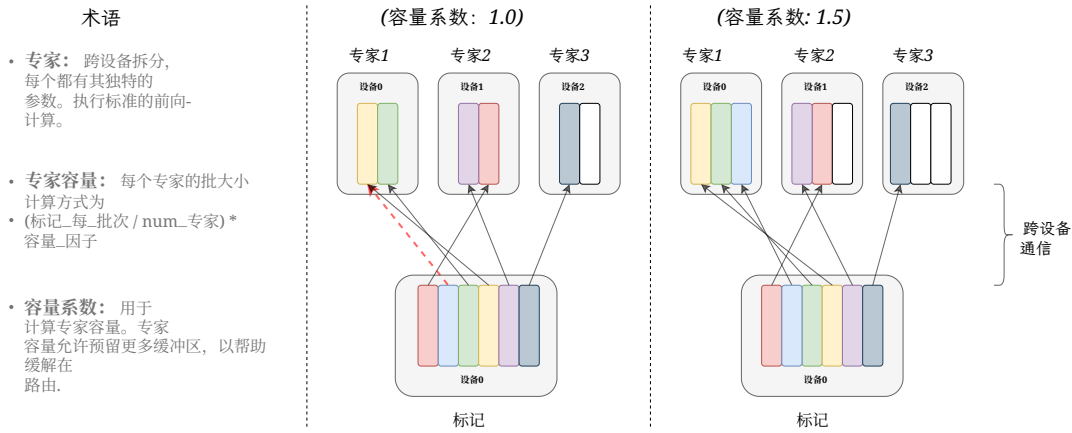


图 3：词元路由动态示意图。每个专家处理由容量系数调节的固定批大小的标记。每个标记被路由到具有最高路由概率的专家，但每个专家的固定批大小为（总标记数 / 专家数量） $\times$ 容量系数。如果标记被不均匀分配，某些专家将发生溢出（以红色虚线表示），导致这些标记不会被本层处理。更大的容量系数可以缓解这种溢出问题，但也会增加计算和通信开销（以填充的白色/空槽表示）。

2.2 高效稀疏路由

我们使用 Mesh-TensorFlow (MTF) (Shazeer 等, 2018 年)，它是一个与 TensorFlow (Abadi 等, 2016 年) 具有相似语义和 API 的库，有助于实现高效的数据并行与模型并行架构。其方式是将物理的一组核心抽象为处理器的逻辑网格。随后，可按命名维度对张量和计算进行分片，从而便于跨维度对模型进行划分。我们在设计模型时考虑到 TPU，其需要静态声明的大小。下面我们介绍我们的分布式的 Switch Transformer 实现。

3. 有关技术说明，请参见第 2.2 节。

分布式 Switch 实现。 我们的所有张量形状都在编译时静态确定，但由于训练和推理中的路由决策，我们的计算是 动态 的。因此，一个重要的技术考量是如何设置专家容量。专家容量——每个专家计算的标记数量——的设置方式是：将批次中的标记数量按专家数量平均分配，然后再按一个容量系数进行扩展，

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}. \quad (3)$$

容量系数大于 1.0 会创建额外的缓冲区，以应对在专家之间标记无法做到完全均衡的情况。若过多的标记被路由到某个专家（下文称为被丢弃的标记），则会跳过计算，并通过残差连接将该标记表示直接传递到下一层。然而，提高专家容量并非没有缺点，因为较高的取值会导致计算和内存浪费。这种权衡在图 3 中进行了说明。经验证据表明，确保较低的被丢弃的标记率对于稀疏专家模型的扩展性很重要。在我们的实验中，我们没有观察到被丢弃的标记数量依赖于专家数量（通常为 $< 1\%$ ）。使用具有足够高系数的辅助负载均衡损失（下一节）能确保良好的负载均衡。我们在表 1 中研究了这些设计决策对模型质量和速度的影响。

可微的负载均衡损失。 为鼓励在专家之间实现负载均衡，我们加入一个辅助损失（Shazeer 等，2017 年，2018 年；Lepikhin 等，2020 年）。如同 Shazeer 等（2018 年）和 Lepikhin 等（2020 年），Switch Transformer 简化了 Shazeer 等（2017 年）中的原始设计，该设计将负载均衡和重要性加权损失分开。对于每个 Switch 层，在训练期间将该辅助损失加入到总模型损失中。给定 N 位专家，其索引从 $i = 1$ 到 N ，以及一个包含 T 个标记的批次 $\mathcal{B}$ ，该辅助损失被计算为向量 f 与 P 之间的缩放点积，

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

其中 f_i 是被分派到专家 i 的标记所占比例，

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \quad (5)$$

并且 P_i 是路由概率中分配给专家 i 的比例，²

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \quad (6)$$

由于我们希望在 N 专家之间对该批次标记进行均匀路由，我们希望这两个向量都具有 $1/N$ 的值。公式 4 的辅助损失鼓励均匀路由，因为它在均匀分布下取到最小值。该目标函数还可以求导为

2. 一个潜在的混淆来源： $p_i(x)$ 是将标记 x 路由到专家 i 的概率。 P_i 是在批次 $\mathcal{B}$ 中跨越 *all tokens*、指向专家 i 的概率比例。

P -向量是可微的，但 f -向量不可微。为了在专家数量变化时保持损失不变，最终损失会乘以专家数量 N ，因为在均匀路由下 $\sum_{i=1}^N (f_i \cdot P_i) = \sum_{i=1}^N (\frac{1}{N} \cdot \frac{1}{N}) = \frac{1}{N}$ 。最后，超参数 α 是这些辅助损失的乘性系数；在整个工作中，我们使用了一个 $\alpha = 10^{-2}$ ，其足够大以确保负载均衡，同时又足够小，不至于淹没主要的交叉熵目标。我们以 10 的幂次扫描了 α 的超参数范围，从 10^{-1} 到 10^{-5} ，并发现 10^{-2} 能够快速实现负载均衡，而不会干扰训练损失。

2.3 综合起来：Switch Transformer

我们对 Switch Transformer 的首次测试，从在“巨量清洁爬取语料库”（C4）上进行预训练开始，该语料库由（Raffel 等，2019 年）提出。我们的预训练目标采用掩码语言建模任务（Taylor, 1953 年；Fedus 等，2018 年；Devlin 等，2018 年），即训练模型预测缺失的标记。在我们的预训练设置中，按 Raffel 等（2019 年）所确定的最优方案，我们丢弃 15% 的标记，然后用一个哨兵标记替换被掩码的序列。为比较我们的模型，我们记录负对数困惑度。⁴ 在本文的所有表格中， $\uparrow$ 表示该指标值越高越好，而 $\downarrow$ 则相反。对本文研究的所有模型的比较见表9。

表 1 展示了 Switch Transformer 和 MoE Transformer（混合专家 Transformer）之间的正面对比。我们的 Switch Transformer 模型与‘T5-Base’（Raffel 等，2019 年）FLOP 匹配（对每个标记的计算量相同）。使用 Top-2 路由的 MoE Transformer（混合专家 Transformer）具有两个专家，每个专家对每个标记分别应用一个独立的 FFN，因此其 FLOPs 更大。所有模型都在相同的硬件上以相同的步数进行训练。请注意，在上述实验设置中，MoE 模型将容量系数从 2.0 降至 1.25 实际上会变慢（840 到 790），这出乎意料。⁵

我们从表 1 中强调三项关键发现：（1）在速度-质量基准上，Switch Transformer 同时优于经过精心调参的稠密模型和 MoE Transformer（混合专家 Transformer）。在给定的计算量和墙钟时间下，Switch Transformer 获得最佳结果。（2）与其 MoE 对应模型相比，Switch Transformer 的计算开销更小。如果我们增大其规模以匹配 MoE Transformer（混合专家 Transformer）的训练速度，我们发现它在按步计上也优于所有 MoE 和稠密模型。（3）在更低的容量系数（1.0、1.25）下，Switch Transformer 表现更好。较小的专家容量表明在大模型场景中，模型内存非常稀缺，此时希望将容量系数尽可能减小。

2.4 改进的训练与微调技术

稀疏专家模型相比标准Transformer可能引入训练困难。由于在这些层中的硬切换（路由）决策，可能导致不稳定性。此外，低精度格式，如 bfloat16（Wang 和 Kanwar, 2019 年），可能会加剧这些问题。

4. 我们在该指标中使用以 e 为底的对数，因此其单位为纳特。5. 请注意，速度测量同时取决于算法和实现细节。Switch Transformer 相较于 MoE（算法）减少了所需的计算，但最终的速度差异会受到底层优化（实现）的影响。

模型	容量因子	之后的质量 100k 步 (↑) (负对数困惑度)	达到质量所需时间 阈值 (↓) (小时)	速度 (↑) (样本/秒)
T5-Base	—	-1.731	未达成 [†]	1600
T5-Large	—	-1.550	131.1	470
MoE-Base	2.0	-1.547	68.7	840
Switch-Base	2.0	-1.554	72.8	860
MoE-Base	1.25	-1.559	80.7	790
Switch-Base	1.25	-1.553	65.0	910
MoE-Base	1.0	-1.572	80.1	860
Switch-Base	1.0	-1.561	62.8	1000
Switch-Base+	1.0	-1.534	67.6	780

表 1: Switch 与 MoE 的基准评测。正面对比, 评估 Switch Transformer 相较于 MoE Transformer 和 T5 稠密基线在每步和每单位时间上的收益。我们通过负对数困惑度以及达到任意选定的质量阈值 (负对数困惑度=-1.50) 所需的时间来衡量质量。所有 MoE 和 Switch Transformer 模型都使用 128 个专家, 且每隔一个前馈层放置专家。对于 Switch-Base+, 我们通过将模型隐藏维度从 768 提升到 896、将注意力头数从 14 提升到 16 来增大模型规模, 直至其速度与 MoE 模型匹配。所有模型都在相同的计算量 (32 核心) 和相同的硬件 (TPUv3) 上进行训练。另需注意, 我们的所有模型都需要超过 100k 步的预训练才能达到我们设定的 -1.50 阈值。† T5-Base 在模型训练的 100k 步内未达到该负对数困惑度。

在我们的路由器的 Softmax 计算中。我们在此描述了训练困难, 以及我们为克服这些困难而采用的方法, 以实现稳定且可扩展的训练。

大型稀疏模型中的选择性精度。 模型不稳定性阻碍了使用高效的 bfloat16 精度进行训练, 因此, Lepikhin 等 (2020) 在其 MoE Transformer (混合专家 Transformer) 中始终使用 float32 精度进行训练。然而, 我们表明, 通过改为选择性地转换为 float32 精度, 在模型的局部部分, 可以获得稳定性, 同时不会产生 float32 张量昂贵的通信开销。这种技术与现代混合精度训练策略一致, 其中模型的某些部分和梯度更新采用更高的精度, 见 Micikevicius 等 (2017)。表2显示, 我们的方法在提供几乎与 bfloat16 训练相同的速度的同时, 赋予了 float32 的训练稳定性。

为此, 我们将路由输入转换为 float32 精度。路由函数以标记为输入, 并生成用于专家计算的选择与重组的分发与合并张量 (详见附录中的代码块15)。重要的是, float32 精度仅在 *within* 路由函数体内——用于该设备上的本地计算。由于在函数末尾会将得到的分发与合并张量重新转换为 bfloat16 精度, 因此不会产生昂贵的 float32 张量

模型 (精度)	质量 (负对数困惑度) ($\uparrow$)	速度 (样本/秒) ($\uparrow$)
Switch-Base (float32)	-1.718	1160
Switch-Base (bfloat16)	-3.780 [发散]	1390
Switch-Base (选择性精度)	-1.716	1390

表2: 选择性精度。我们将本地路由操作转换为 float32, 同时在其他地方保持 bfloat16 精度, 以在稳定我们的模型的同时, 获得与 (不稳定的) bfloat16 精度训练几乎相同的速度。我们在训练早期于固定步数后, 测量32专家模型的质量, 并评估其速度性能。对于使用 float32 的 Switch-Base 和采用 选择性预见 的情况, 我们观察到类似的学习动态。

通过全对全通信操作进行广播, 但我们仍然受益于 float32 更高的稳定性。

较小的参数初始化以提高稳定性。恰当的初始化对深度学习中的成功训练至关重要, 我们尤其观察到这对 Switch Transformer 更为重要。我们通过从均值为 $\mu = 0$ 和标准差为 $\sigma = \sqrt{s/n}$ 的截断正态分布中采样来初始化我们的权重矩阵, 其中, s 是一个尺度超参数, n 是权重张量中的输入单元数量 (例如, 扇入)。⁶

作为对不稳定性的额外补救, 我们建议将默认的 Transformer 初始化尺度 $s = 1.0$ 按 10 的因子降低。这样既提高了质量, 又在我们的实验中降低了出现不稳定的训练的可能性。表3衡量了在训练早期模型质量的提升和方差的降低。我们发现

模型 (初始化尺度)	平均质量 (负对数困惑度)	质量的标准差 (负对数困惑度)
Switch-Base (0.1x-init)	-2.72	0.01
Switch-Base (1.0x-init)	-3.60	0.68

表3: 降低初始化尺度提升稳定性。降低初始化尺度会带来更好的模型质量, 并使 Switch Transformer 的训练更加稳定。这里我们记录了一个32专家模型在 3.5k 步数后的模型质量 (以负对数困惑度量) 的平均值和标准差 (每种情况下使用 3 个随机种子)。

以负对数困惑度衡量的平均模型质量显著提升, 且跨多次运行的方差大幅降低。此外, 这一相同的初始化方案对跨越多个数量级的模型都普遍有效。我们使用相同的方法, 从小至我们的2.23亿参数基线模型, 到超过一万亿参数的巨型模型, 都能稳定地进行训练。

6. 超过均值两个标准差的值将被重新采样。

正则化大型稀疏模型。 本文考虑自然语言处理中的常见方法：在大型语料库上进行预训练，随后在更小的下游任务（如摘要生成或问答）上进行微调。一个自然而然出现的问题是过拟合，因为许多微调任务的样本非常少。在标准的 Transformer 模型的微调过程中，Raffel 等（2019 年）在每一层使用 Dropout（随机失活）（Srivastava 等人（2014））来防止过拟合。我们的 Switch Transformer 的参数显著多于 FLOP 对齐的致密基线，这可能会在这些较小的下游任务上导致更严重的过拟合。

模型 (Dropout (随机失活))	GLUE 基准 CNNDM 数据集 SQuAD SuperGLUE			
T5-Base (d=0.1)	82.9	19.6	83.5	72.4
Switch-Base (d=0.1)	84.7	19.1	83.7	73.0
Switch-Base (d=0.2)	84.4	19.2	83.9	73.2
Switch-Base (d=0.3)	83.9	19.6	83.4	70.7
Switch-Base (d=0.1, ed=0.4)	85.2	19.6	83.7	73.0

表4：微调正则化结果。对在 C4 数据集上以 34B 个标记进行预训练的 Switch Transformer 模型进行微调时，遍历不同的丢弃率设置（数值越高越好）。我们观察到：在所有非专家层使用较低的标准丢弃率，并在专家前馈层使用更大的丢弃率，表现最佳。

因此，我们提出一种在微调期间缓解该问题的简单方法：增大专家内部的 Dropout（随机失活），我们将其称为专家 *dropout*。在微调时，我们仅在每个专家层的中间前馈计算处显著提高丢弃率。表4给出了我们的专家 *dropout* 策略的结果。我们观察到，简单地在所有层上增加 Dropout（随机失活）会带来更差的性能。然而，在非专家层设置较小的丢弃率（0.1），并在专家层设置更大的丢弃率（0.4），可在四个较小的下游任务上带来性能提升。

3. 扩展性特性

我们呈现了一项关于扩展性特性的 Switch Transformer 架构在预训练期间的研究。根据 Kaplan 等（2020），我们考虑一种情形，其中模型既不受计算预算也不受数据量的限制。为避免数据瓶颈，我们使用规模庞大的 C4 语料库，其中包含超过 180B 个目标标记（Raffel 等，2019 年），并训练直至观察到递减收益。

专家数量是扩展我们模型最有效的维度。增加专家数量会使计算成本大致保持不变，因为无论可供选择的专家有多少，模型对每个标记都只选择一个专家。路由器必须针对更多专家计算概率分布，然而这是一种轻量级计算，其成本为 $O(d_{model} \times \text{专家数量})$ ，其中 d_{model} 是嵌入维度的

在层之间传递的标记。在本节中，我们在固定计算预算下，以步数为基准和以时间为基准来考虑扩展性特性。

3.1 基于步数的缩放结果

图4展示了在将所有模型训练固定步数的情况下，随着专家数量增加而带来的持续扩展性收益。我们观察到一个清晰趋势：当保持每个标记的FLOPs固定时，拥有更多参数（专家）会加快训练。左图展示了在稀疏模型参数与测试损失之间（在固定每个标记的FLOPs下）一致的扩展性特性。这揭示了沿着稀疏模型参数这一额外维度进行扩展的优势。我们的右图评估了一个稠密模型变体和四个FLOP匹配的稀疏变体的样本效率。我们发现，增加专家数量会带来样本效率更高的模型。我们的 Switch-Base 64专家模型在60k步就达到了 T5-Base 模型在450k步的相同性能，这在每步时间方面带来了7.5倍加速。此外，与 Kaplan 等（2020年）的发现一致，我们发现更大的模型也更样本效率高——在观测标记数固定的情况下学习得更快。

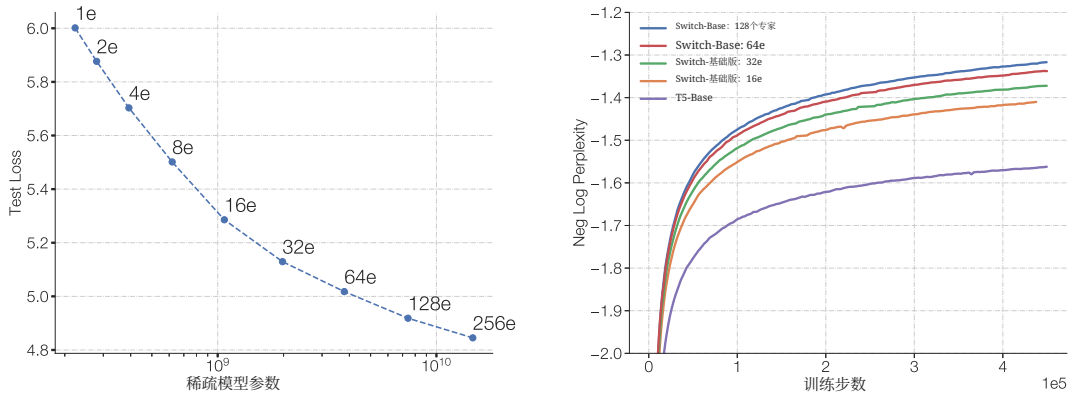


图4: Switch Transformer 的扩展性特性。左图：我们用困惑度来衡量质量提升，并在通过扩展专家数量增加参数规模时进行评估。左上角点对应于具有2.23亿参数的 T5-Base 模型。从左上到右下，我们将专家数量依次加倍，从 2、4、8 等，一直到右下角点的 256 专家模型，具有147亿参数。尽管所有模型使用了相同的计算预算，我们仍观察到随着专家数量扩展而带来的一致改进。右图：每步负对数困惑度，针对不同的专家数量进行扫描。稠密基线用紫色曲线表示，我们注意到我们的 Switch-Base 模型具有更高的样本效率。

3.2 基于时间的扩展结果

图4表明，以步为基准，随着我们增加专家数量，性能持续提升。虽然我们的模型与基线的每个标记的FLOPs大致相同，但我们的 Switch Transformer 模型会带来跨设备的额外通信开销，以及路由机制的额外计算。因此，在以步数为基准观察到的样本效率提升，并不一定能转化为按挂钟时间衡量的更好模型质量。这引出了一个问题：

在固定的训练时长和计算预算下，应该训练稠密模型还是稀疏模型？

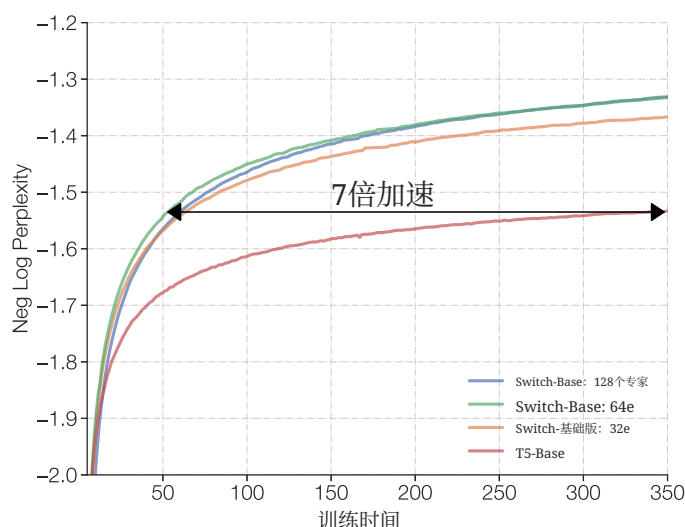


图5: Switch Transformer 的速度优势。所有模型均在 32个TPUv3核心 上训练，且每个样本的FLOPs相同。在固定的计算和训练时间下，Switch Transformer 显著优于稠密 Transformer 基线。我们的 64个专家的Switch-Base模型在七分之一T5-Base 的时间内达到相同的质量，并且继续提升。

图5和图6回答了这个问题。图5将预训练模型质量作为时间的函数进行度量。在固定的训练时长和计算预算下，Switch Transformer 带来显著加速。在此设置下，我们的 Switch-Base 64专家模型的训练仅需七分之一 T5-Base 达到相似困惑度所需的时间。

3.3 与更大稠密模型的扩展对比

上述分析表明，计算量匹配的稠密模型不及其 Switch 对应模型。图6考虑了另一种情形：如果我们把资源分配给一个更大的稠密模型会怎样？我们现在就这样做，将 Switch-Base 与下一强基线 T5-Large 进行比较。但尽管 T5-Large 每个标记的FLOPs多3.5倍，

Switch-Base 仍然样本效率高，并带来2.5倍加速。此外，只需设计一个新的、更大的稀疏版本 Switch-Large，与 T5-Large FLOP 匹配，就能获得更多提升。我们这样做，并在下文展示了更好的扩展性和微调。

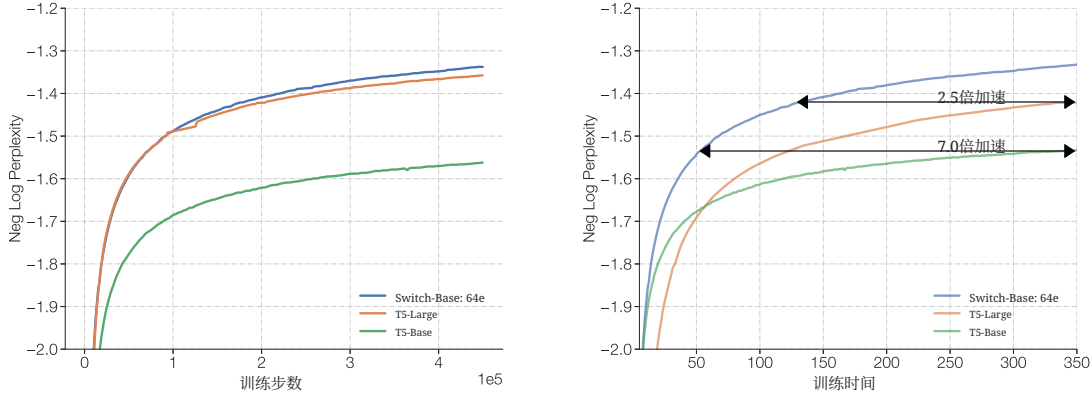


图6：使用 Switch 层或采用标准稠密模型扩展来扩展 Transformer 模型。左图：Switch-Base 比 T5-Base 和 T5-Large 变体样本效率更高，后者每个标记的FLOPs多3.5倍。右图：与之前一样，在墙钟时间基准上，我们发现 Switch-Base 仍然更快，并且相较于 T5-Large 带来2.5倍加速。

4. 下游结果

第3节在预训练期间展示了更优的扩展性特性，但我们现在验证这些增益会转化为下游任务上的语言学习能力提升。我们首先在多样化的自然语言处理任务上进行微调。接着，我们研究通过将其蒸馏为小型——且易于部署的——稠密基线，将稀疏模型的内存占用降低90%以上。最后，我们在多任务、多语言设置下对这些改进进行测量，表明 Switch Transformer 是强大的多任务学习者，在全部101种语言上优于多语言 T5-base 模型。

4.1 微调

基线和 Switch 模型用于微调。 我们的基线是经过高度调优的 2.23亿参数 T5-Base 模型和 7.39亿参数 T5-Large 模型 (Raffel 等, 2019 年)。对于这两个版本，我们设计了 FLOP 匹配的 Switch Transformer，具有更多参数，相关内容汇总于表9。⁷ 我们的基线与 Raffel 等 (2019 年) 的有所不同，因为我们在改进的 C4 语料库上进行预训练，该语料库移除了样本内文本重复，从而提高了其作为预训练任务的有效性，Lee 等

7. FLOPs 按 Kaplan 等 (2020) 的方法，仅按前向传播进行计算。

(2021年)。在我们的协议中，我们预训练 with 2^{20} (1,048,576) 每批次标记数，持续 55 万步，总标记数为 576B。随后，我们在多样化任务集上进行微调，除 Switch 层外的所有层使用 0.1 的丢弃率，Switch 层使用 0.4 的丢弃率（见表4）。我们使用 1M 的批大小进行微调，持续 16k 步，并且针对每个任务，我们每 200 步评估一次模型质量，并报告在验证集上计算得到的峰值性能。

微调任务与数据集。 我们选择用于考察语言能力的任务，包括问答、摘要生成以及世界知识。语言基准 GLUE (Wang 等人, 2018年) 和 SuperGLUE (Wang 等人, 2019年) 被作为复合混合来处理，所有任务按各自所含标记数量的比例进行混合。这些基准包含需要情感分析 (SST-2 数据集)、词义消歧 (WIC 数据集)、句子相似度 (MRPC 数据集、STS-B 数据集、QQP 数据集)、自然语言推理 (MNLI 数据集、QNLI 数据集、RTE 数据集、CB 数据集)、问答 (MultiRC 数据集、RECORD 数据集、BoolQ 数据集)、共指消解 (WNLI 数据集、WSC 数据集)、句子补全 (COPA 数据集) 以及句子可接受性 (CoLA 数据集) 的任务。CNNDM (Hermann 等人, 2015年) 和 BBC XSum (Narayan 等人, 2018年) 数据集用于衡量对文章的摘要生成能力。我们使用 SQuAD (Rajpurkar 等人, 2016年) 数据集和 ARC 推理挑战 (Clark 等人, 2018年) 来考察问答。此外，与 Roberts 等人 (2020年) 一致，我们通过在三个闭卷问答数据集上进行微调来评估模型的知识：Natural Questions (Kwiatkowski 等人, 2019年)、Web Questions (Berant 等人, 2013年) 和 Trivia QA (Joshi 等人, 2017年)。所谓闭卷，是指在没有任何补充参考或上下文材料的情况下提出的问题。为衡量模型的常识推理能力，我们在 Winogrande 模式挑战 (Sakaguchi 等人, 2020年) 上对其进行评估。最后，我们在对抗性 NLI 基准 (Nie 等人, 2019年) 上测试模型的自然语言推理能力。

微调指标。 本文在全文中使用以下评估指标：我们在 GLUE 基准和 SuperGLUE 上报告所有子任务的平均得分。在 CNNDM 数据集和 XSum 数据集上使用 ROUGE-2 指标。在 SQuAD 和闭卷式任务 (Web、Natural 和 Trivia Questions) 中，我们报告与目标完全匹配的答案所占百分比（有关该度量的更多细节及其不足之处，参见 Roberts 等人 (2020年)）。最后，在 ARC Easy 数据集、ARC Challenge 数据集、ANLI 和 Winogrande 数据集上，我们报告生成回答的准确率。

微调结果。 我们在许多自然语言任务上观察到显著的下游改进。来自 SuperGLUE 的改进尤为突出，我们发现，FLOP 匹配的 Switch 变体相较于 T5-Base 和 T5-Large 基线分别提升了 4.4 和 2 个百分点，同时在 Winogrande 数据集、闭卷式 Trivia QA 数据集以及 XSum 数据集上也有大幅提升。⁸ 在我们的微调研究中，唯一未观察到增益的任务是 AI2 推理挑战 (ARC) 数据集，其中 T5-Base 在挑战数据集上优于 Switch-Base，而 T5-Large 在简单数据集上优于 Switch-Large。总体而言，我们在涵盖推理与知识密集型任务上观察到显著的改进。这验证了我们的架构，它不仅在预训练阶段表现出色，还能通过微调将质量提升转移到下游任务。

8. 我们的 T5 和 Switch 模型在修订版 C4 数据集上进行预训练，以每批次标记数为 2^{20} ，训练 55 万步，以实现公平比较。

模型	GLUE	SQuAD	SuperGLUE	Winogrande (XL) 数据集
T5-Base	84.3	85.5	75.1	66.6
Switch-Base	86.7	87.2	79.5	73.3
T5-Large	87.8	88.1	82.7	79.1
Switch-Large	88.5	88.6	84.7	83.0

模型	XSum	ANLI (R3) 数据集	ARC Easy 数据集	ARC 挑战版
T5-Base	18.7	51.8	56.7	35.5
Switch-Base	20.3	54.0	61.3	32.8
T5-Large	20.9	56.6	68.8	35.5
Switch-Large	22.3	58.6	66.0	35.5

模型	闭卷网页问答	闭卷自然问题问答	闭卷琐事问答
T5-Base	26.6	25.8	24.5
Switch-Base	27.4	26.8	30.7
T5-Large	27.7	27.6	29.5
Switch-Large	31.3	29.5	36.9

表5: 微调结果。T5 基线和 Switch 模型在多种自然语言测试 (验证集; 数值越高越好) 上的微调结果。我们将 FLOP 匹配的 Switch 模型与 T5-Base 和 T5-Large 基线进行比较。在所考虑的大多数任务上, 我们发现 Switch 变体都有显著提升。我们在两种模型规模上以及在推理与知识密集型语言任务上均观察到增益。

4.2 蒸馏

部署拥有数十亿或数万亿参数的海量神经网络并不方便。为缓解这一问题, 我们研究蒸馏 (Hinton 等, 2015) 大型稀疏模型为小型稠密模型。未来工作还可以研究将大规模模型蒸馏为更小的 *sparse* 模型。

蒸馏技术。 在表6中, 我们研究了多种蒸馏技术。这些技术基于 Sanh 等 (2019), 他们研究了针对 BERT 模型的蒸馏方法。我们发现, 用非专家权重初始化稠密模型可带来适度的改进。这是可行的, 因为所有模型都是 FLOP 匹配的, 因此非专家层将具有相同的维度。由于在 Transformer 中, 专家层通常只在每个或每隔一个 FFN 层处添加, 这使得许多权重可以用训练好的参数进行初始化。此外, 我们观察到采用 0.25 的教师概率与 0.75 的真实标签的混合可带来蒸馏改进。通过结合这两种技术, 我们在仅使用 $\approx 1/20^{th}$ 的参数, 保留了来自更大稀疏模型的 $\approx 30\%$ 质量提升。质量增益是指

Switch-基础版（教师）与 T5-基础版（学生）之间的质量差异。因此，质量增益为 100% 意味着学生的性能等于教师。

技术	参数	质量 (↑)
T5-Base	223M	-1.636
Switch-Base	3,800M	-1.444
蒸馏	223M	(3%) -1.631
+从教师初始化非专家权重	223M	(20%) -1.598
+ 0.75 硬损失与软损失的混合	223M	(29%) -1.580
初始化基线（无蒸馏）		
从教师初始化非专家权重	223M	-1.639

表6：用于语言建模的Switch Transformer蒸馏。用来自Switch-Base的非专家权重初始化T5-Base，并使用由教师模型与真实标签混合而成的损失，可以获得最佳性能。我们可以把参数量多100倍的大型稀疏模型的性能提升中的30%蒸馏回一个小型稠密模型。作为最终的基线，我们发现：用专家权重初始化的T5-Base，如果在没有蒸馏的情况下按常规训练，并不会带来改进。

可实现的压缩率。 使用我们在表6中描述的最佳蒸馏技术，我们将多种稀疏模型蒸馏为稠密模型。我们蒸馏了Switch-Base版本，遍历递增的专家数量，对应于在11亿到147亿参数之间变化。通过蒸馏，我们在压缩82%的同时，能够保留11亿参数模型的37%质量增益。在极端情况下，当我们将模型压缩99%时，我们仍然能够保持教师模型质量提升的28%。

对微调模型进行蒸馏。 我们最后通过一项研究作结，探讨如何将一个已微调的稀疏模型蒸馏为稠密模型。表8显示了将一个在 SuperGLUE 任务上微调的、具有74亿参数的 Switch-Base 模型蒸馏到 2.23亿参数的 T5-Base 的结果。与我们的预训练结果类似，我们发现，当蒸馏到一个FLOP匹配的稠密变体时，能够保留稀疏模型30%的增益。一个潜在的未来方向（本文未予探讨）是考察用于微调任务的特定专家，并将其提取出来，以实现更好的模型压缩。

4.3 多语种学习

在最后一组下游实验中，我们在由101种语言组成的混合数据上进行预训练，衡量模型质量与速度之间的权衡。我们基于 mT5 (Xue 等, 2020年) 的最新工作进行构建和基准评测，该工作是 T5 的多语种扩展。我们在 mT5 中引入的 Common Crawl 数据集的多语言变体 (mC4) 上进行预训练，覆盖101种语言，但由于某些语言存在文字变体，该混合包含107个任务。

在图 7 中，我们绘制了 FLOP 匹配的 Switch 模型 mSwitch-Base 相对于 T5 基础变体 mT5-Base 在所有语言上的负对数困惑度的质量提升。之后

	稠密	稀疏				
参数	223M	1.1B	2.0B	3.8B	7.4B	147亿
预训练 负对数困惑度 ($\uparrow$)	-1.636	-1.505	-1.474	-1.444	-1.432	-1.427
蒸馏 负对数困惑度 ($\uparrow$)	—	-1.587	-1.585	-1.579	-1.582	-1.578
教师绩效百分比	—	37%	32%	30 %	27 %	28 %
压缩百分比	—	82 %	90 %	95 %	97 %	99 %

表7: 蒸馏压缩率。我们在将大型稀疏模型蒸馏到稠密基线时评估其质量。我们的基线 T5-Base 的负对数困惑度质量为 -1.636。在右侧列中, 我们随后将越来越大的稀疏模型蒸馏到同一架构中。通过结合权重初始化以及硬损失与软损失的混合, 我们可以将稀疏教师模型缩小 95%+同时保留 30% 的质量增益。然而, 对于明显更好且更大的预训练教师模型, 我们预计需要更大的学生模型才能达到这些压缩率。

模型	参数	FLOPs	SuperGLUE ($\uparrow$)
T5-Base	223M	124B	74.6
Switch-Base	7410M	124B	81.3
蒸馏版 T5-Base	223M	124B	(30%) 76.6

表8: 蒸馏一个微调的 SuperGLUE 模型。我们将一个在 SuperGLUE 任务上微调的 Switch-Base 模型蒸馏到一个 T5-Base 模型上。我们观察到, 在更小的数据集上, 我们的大型稀疏模型可以作为蒸馏的有效教师模型。我们发现, 我们再次在一个被压缩 97% 的模型上达到了教师模型性能的 30%。

对两个版本进行100万步的预训练后, 我们发现, 在所有 101种语言上, Switch Transformer 相比基线提高了最终负对数困惑度。在图8中, 我们给出另一种视角, 将使用 Switch Transformer 相对于 mT5-Base 的每步加速比 绘制为直方图。⁹ 我们发现, 相对于 mT5-Base 的均值加速比为5倍, 并且91%的语言至少达到4倍加速。这表明 Switch Transformer 是有效的多任务和多语言学习者。

5. 使用数据并行、模型并行和专家并行设计模型

随意增加专家数量会遭遇递减收益 (图4)。在此我们描述互补的 扩展策略。扩展 Transformer 的常见方式是同步增加维度, 例如 d_{model} 或 d_{ff} 。这会同时增加参数

9. 以步为基准的加速比计算为: 基线的步数除以我们的模型达到相同质量所需的步数。

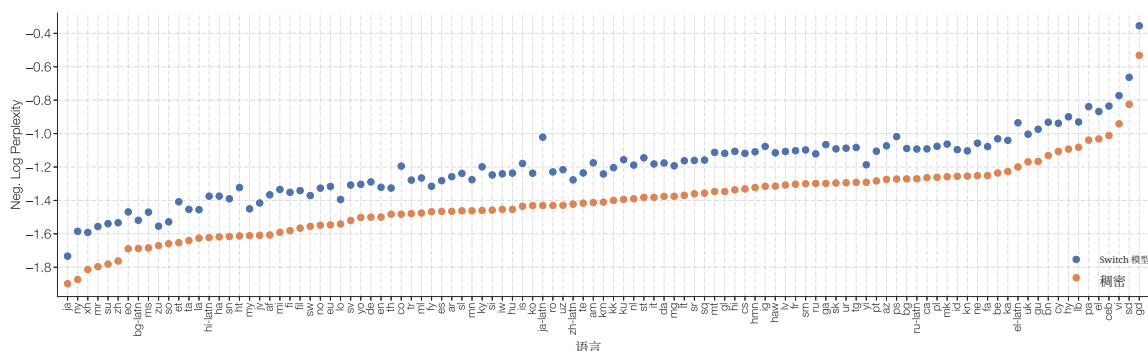


图 7：在101种语言上的多语言预训练。对101种语言进行多任务训练时，Switch T5 Base 模型相对于稠密基线的改进。我们观察到 Switch Transformer 在多任务训练设置中表现良好，并在全部101种语言上都带来改进。

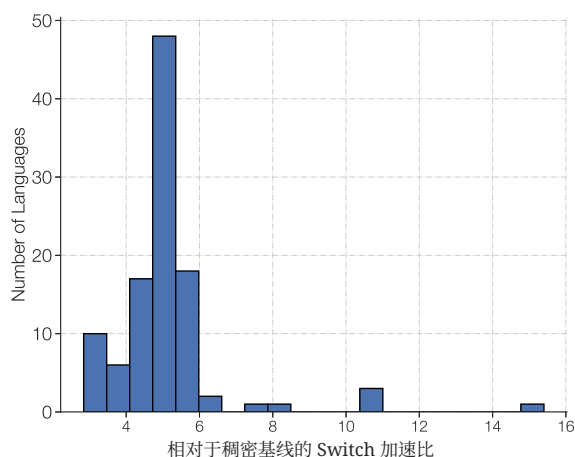


图8：在101种语言上的多语言预训练。我们为每种语言绘制了直方图，展示 Switch Transformer 相对于 FLOP 匹配的 T5 稠密基线在达到相同质量时的步数加速比。在全部 101种语言上，我们相对于 mT5-Base 的平均步数加速比为5倍；对于91%的语言，我们记录到为达到 mT5-Base 的最终困惑度所需的加速比为4 倍或更高。

以及所执行的计算，并最终受每个加速器的内存限制。一旦超过加速器内存的大小，就可以采用单程序多数据（SPMD）模型并行。本节研究组合数据并行、模型并行和专家并行的权衡。

回顾前馈网络（FFN）层。我们以 FFN 层为例，说明在 Mesh TensorFlow（Shazeer 等，2018 年）中数据、模型和专家并行如何工作，并在此作简要回顾。我们假设批次中有 B 个标记，每个的维度为

d_{model} FFN 的输入 (x) 和输出 (y) 的大小均为 $[B, d_{model}]$, 中间表示 (h) 的大小为 $[B, d_{ff}]$, 其中 d_{ff} 通常比 d_{model} 大数倍。在 FFN 中, 中间表示是 $h = xW_{in}$, 然后该层的输出是 $y = ReLU(h)W_{out}$ 因此, W_{in} 和 W_{out} 独立地应用于每个标记, 其大小分别为 $[d_{model}, d_{ff}]$ 和 $[d_{ff}, d_{model}]$ 。

我们描述分区的两个方面: 权重 和数据批次 如何在核心之间划分, 如图9所示。我们将所有可用的核心表示为 N , 然后网格 TensorFlow 可以将其重新映射为处理器的逻辑多维网格。这里我们创建一个二维逻辑网格, 其中一个维度表示用于数据并行分片的方式

(n), 另一个维度表示模型并行分片 (m)。总核心数必须等于数据并行和模型并行分片方式的乘积, 例如 $N = n \times m$ 。为了在核心之间对该层进行分片, 包含该批次 B 个标记的张量将在 n 个数据并行核心之间分片, 因此每个核心包含 B/n 个标记。具有 d_{ff} 的张量和变量随后在 m 个模型并行核心之间分片。对于带有专家层的变体, 我们考虑 E 个专家, 每个专家最多可处理 C 个标记。

Term	描述
B	该批次中的标记数量。
N	总核心数。
n	数据并行划分数。
m	模型并行划分数。
E	Switch 层中的专家数量。
C	专家容量, 即每个专家的批大小。

5.1 数据并行

在训练数据并行模型时, 这通常是分布式训练的标准做法, 此时所有核心都分配给数据并行维度或 $n = N, m = 1$ 。其优点是, 在整个前向和反向传播完成之前不需要任何通信, 然后将梯度在所有核心之间聚合。这对应于图9最左侧的列。

5.2 模型并行

我们现在考虑一种情形: 所有核心都专用于模型并行维度, 因此 $n = 1, m = N$ 。现在所有核心都必须保存完整的 B 标记, 并且每个核心都包含权重的唯一切片。每次前向和反向传播都会产生通信开销。每个核心会发送一个大小为 $[B, d_{model}]$ 的张量用于计算第二次矩阵乘法 $ReLU(h)W_{out}$, 因为 d_{ff} 维度被分片并且必须进行求和。一般而言, 只要需要对跨核心分片的维度进行求和, 那么就需要在前向和反向传播中各加入一次全规约操作。这与纯数据并行形成对比, 在纯数据并行中, 全规约只会发生在整个前向和反向传播结束时。

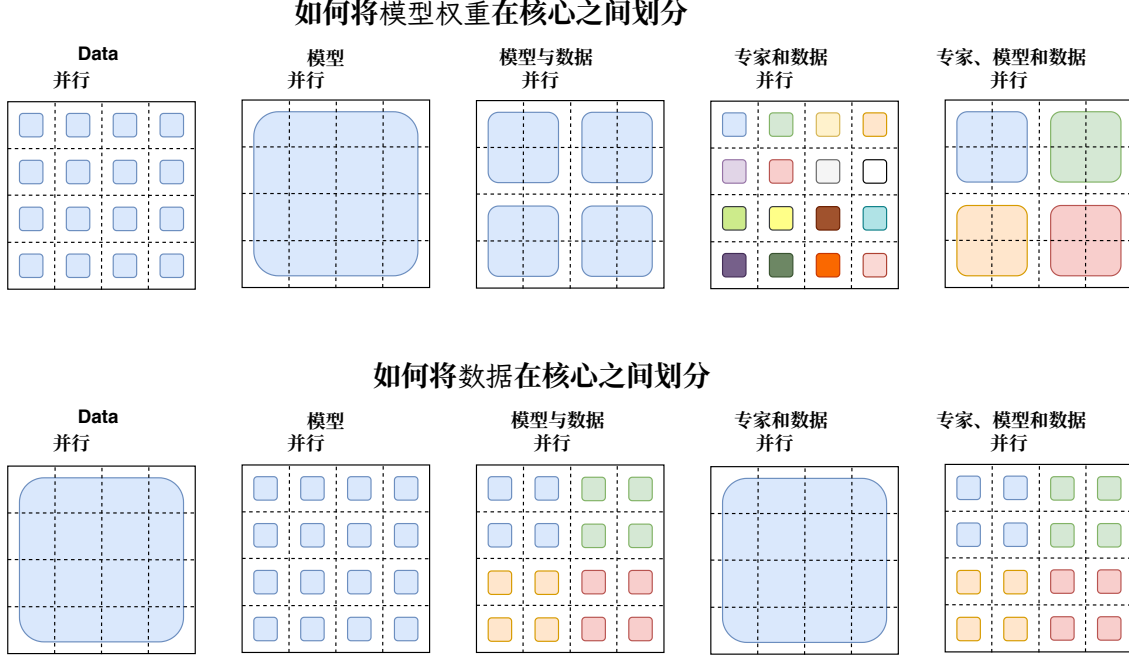


图9：数据与权重分区策略。每个 4×4 虚线网格表示16个核心，阴影方格表示该核心上包含的数据（要么是模型权重，要么是标记批次）。我们展示了在每种策略下模型权重和数据张量是如何被划分的。**第一行：**展示模型权重如何在核心之间划分。本行中不同大小的形状代表前馈网络（FFN）层中更大的权重矩阵（例如更大的 d_{ff} 尺寸）。阴影方格的每种颜色对应一个唯一的权重矩阵。参数数量每个核心是固定的，但更大的权重矩阵会对每个标记进行更多计算。**第二行：**展示数据批次如何在核心之间划分。每个核心持有相同的标记数量，从而在所有策略中保持固定的内存使用。不同的分区策略具有不同的性质：允许每个核心在跨核心时要么拥有相同的标记，要么拥有不同的标记，这正由不同的颜色加以表示。

5.3 模型与数据并行

对于大规模模型，同时混合使用模型并行和数据并行是常见做法，最大规模的 T5 模型（Raffel 等，2019 年；Xue 等，2020）以及 GPT-3（Brown 等，2020 年）都采用了这种方式。在共有 $N = n \times m$ 核心的情况下，现在每个核心将负责 B/n 标记，以及 d_{ff}/m 的权重和中间激活。在前向和反向传播中，每个核心都会在一次全规约操作中通信一个大小为 $[B/n, d_{model}]$ 的张量。

5.4 专家并行与数据并行

接下来我们描述专家并行与数据并行的划分策略。**Switch Transformer** 会将其所有核心分配给数据划分维度 n ，这也将对应于模型中的专家数量。对于每个核心的每个标记，路由器在本地计算到各专家的分配。输出为大小为 $[n, B/n, E, C]$ 的二进制矩阵，该矩阵沿第一维划分，并决定专家分配。然后使用该二进制矩阵与形状为 $[n, B/n, d_{model}]$ 的输入张量进行矩阵乘法，以实现聚合。

$$\text{einsum}([n, B/n, d_{model}], [n, B/n, E, C], \text{dimension} = [B/n]) \quad (7)$$

从而得到形状为 $[n, E, C, d_{model}]$ 的最终张量，并在第一维上进行分片。由于每个核心都有其自己的专家，我们执行大小为 $[E, C, d_{model}]$ 的全到全通信，以便现在对 E 维度进行分片，而不是对 n 维度分片。在前向传播中，还会有大小为 $E \times C \times d_{model}$ 的 **bfloat16** 张量的额外通信开销，用于以类似方式从位于不同核心上的各个专家接收标记。有关专家划分代码的详细分析，参见附录F。

5.5 专家并行、模型并行与数据并行

在我们最佳模型的设计中，我们致力于在每个标记的FLOPs与参数数量之间取得平衡。随着专家数量的扩展，我们会增加参数数量，但不会改变每个标记的FLOPs。为了提高FLOPs，我们还必须增加 d_{ff} 维度（这也会增加参数，但增速较慢）。这带来了权衡：当我们增加 d_{ff} 时，每核内存将耗尽，从而需要增加 m 。但由于我们的核心总数固定为 N ，且 $N = n \times m$ ，我们必须减少 n ，这迫使我们使用更小的批大小（以保持每核标记数不变）。

当同时采用模型并行和专家并行时，路由标记到正确专家会产生全对全通信开销，同时模型并行内部还会带来全规约通信。将这三种方法结合时，在FLOPs、通信开销与每核内存之间寻求平衡会变得相当复杂，最佳映射方案需要通过经验确定。关于专家数量如何影响下游性能，参见我们在第5.6节中的进一步分析。

5.6 迈向万亿参数模型

结合专家并行、模型并行与数据并行，我们设计了两个大型 **Switch Transformer** 模型，参数规模分别为3950亿和1.6万亿参数。我们研究这些模型在作为语言模型进行上游预训练时的表现，以及它们的下游微调性能。这两个不同模型参数、每个序列的FLOPs以及超参数列于下方的表9。我们描述了 **Transformer** 的标准超参数，包括 d_{model} 、 d_{ff} 、 d_{kv} 、注意力头数和层数；还介绍了一个不太常见的特性 FFN_{GEGLU} ，它指的是FFN层的一种变体：用两组进行非线性组合的权重替代扩展矩阵（Shazeer, 2020年）。

如第5.4节前述，**Switch-C** 模型仅采用专家并行而不使用模型并行。因此，控制宽度的超参数，

模型	参数	每序列浮点运算量	d_{model}	FFN_{GEGLU}	d_{ff}	d_{kv}	注意力头数
T5-Base	0.2B	124B	768	✓	2048	64	12
T5-Large	0.7B	425B	1024	✓	2816	64	16
T5-XXL	11B	6.3T	4096	✓	10240	64	64
Switch-Base	7B	124B	768	✓	2048	64	12
Switch-Large	26B	425B	1024	✓	2816	64	16
Switch-XXL	395B	6.3T	4096	✓	10240	64	64
Switch-C	15710亿	890B	2080		6144	64	32

模型	专家频率	层数	专家数量	负对数困惑度 @250k	负对数困惑度 @ 500k
T5-Base	—	12	—	-1.599	-1.556
T5-Large	—	24	—	-1.402	-1.350
T5-XXL	—	24	—	-1.147	-1.095
Switch-Base	1/2	12	128	-1.370	-1.306
Switch-Large	1/2	24	128	-1.248	-1.177
Switch-XXL	1/2	24	64	-1.086	-1.008
Switch-C	1	15	2048	-1.096	-1.043

表9: Switch模型设计与预训练性能。我们将 T5 模型的超参数和预训练性能与我们的 Switch Transformer 变体进行比较。最后两列分别记录在 C4 数据集上经过25万步和50万步后的预训练模型质量。我们观察到, Switch-C Transformer 变体在达到固定困惑度(在相同计算预算下)时比 T5-XXL 模型快 4 倍, 且随着训练的推进, 这一差距还在扩大。

深度、注意力头数等都远小于 T5-XXL 模型。相较之下, Switch-XXL 在 FLOP 上与 T5-XXL 模型匹配, 这使得可以使用更大的超参数维度, 但代价是由模型并行引入的额外通信开销(更多细节见第5.5节)。

样本效率对比 T5-XXL。在表9的最后两列中, 我们分别记录了在 C4 语料库上于 25万步 和 50万步后的负对数困惑度。在 25万步 时, 我们发现两种 Switch Transformer 变体相较于 T5-XXL 版本的负对数困惑度提升了超过 0.061。¹⁰ 为便于理解 0.061 这一差距的意义, 我们注意到 T5-XXL 模型不得不再训练额外的 25万步才提升 0.052。随着进一步训练, 这一差距持续扩大; 到 50万步 时, Switch-XXL 模型相较于 T5-XXL 高出 0.087。

训练不稳定性。然而, 正如引言中所述, 大型稀疏模型可能不稳定, 随着规模的扩大, 我们会遇到一些零星的问题。我们发现, 更大的 Switch-C 模型, 具有1.6万亿参数和 2048个专家, 完全没有出现训练不稳定性。相反, Switch XXL 版本, 其每个序列的 FLOPs 近乎增大10倍, 有时会不稳定。因此, 尽管这是我们以步数为基准更好的模型, 我们并未进行完整的100万步预训练, 这与 T5 (Raffel 等, 2019 年) 最终报告的结果保持一致。

10. 这里报告的质量差异是下界, 实际可能更大。T5-XXL 预训练于更容易的 C4 数据集, 该数据集在样本中包含重复的、因此容易被复制的片段。

推理微调性能。 作为对模型质量的初步评估，我们使用在503B 个标记上部分预训练的 Switch-XXL 模型，这大约是 T5-XXL 模型所用文本的一半。使用该检查点，我们为了效率进行多任务训练，所有任务联合学习，而不是分别微调。我们发现，在验证集上的 SQuAD 准确率提高到 89.7，而当前最先进水平为 91.3。接着，SuperGLUE 的平均测试得分为 87.5；相比之下，T5 版本得到 89.3，当前最先进水平为 90.0（Wang 等人，2019 年）。在 ANLI（Nie 等人，2019 年）上，Switch XXL 超过此前的当前最先进水平，取得了 65.7 的准确率，而此前最佳为 49.4（Yang 等人，2020 年）。我们注意到，尽管 Switch-XXL 在上游预训练任务上的负对数困惑度达到当前最先进水平，但这些提升尚未完全转化为 SOTA 下游性能。我们在附录E中对此问题进行了进一步研究。

基于知识的微调性能。 最后，我们还通过三个闭卷知识型任务（Natural Questions、WebQuestions 和 TriviaQA）对模型的知识进行了早期考察，且未使用显著片段掩蔽（Guu 等，2020年）进行额外预训练。在这三种情况下，相比先前的当前最先进水平的 T5-XXL 模型（未使用 SSM），我们都有所提升。Natural Questions 的精确匹配由先前最佳的 32.8 提高到 34.4，Web Questions 由 37.2 提高到 41.0，TriviaQA 由 42.9 提高到 47.5。

总而言之，尽管训练所用数据不到其他模型的一半，我们已经得到可比的，且有时达到当前最先进水平的模型质量。目前，Switch Transformer 更好地将显著的上游增益转移到知识型任务上，而非推理任务（见附录E）。从大型专家模型中挖掘更强的微调性能是一个活跃的研究问题，而预训练困惑度表明未来仍有改进空间。

6. 相关工作

神经网络中规模的重要性已被广泛认可，并已提出多种方法。近期工作通过使用模型并行（例如将权重和张量拆分到多个核心）把模型扩展到数十亿参数（Shazeer 等，2018 年；Rajbhandari 等，2019 年；Raffel 等，2019 年；Brown 等，2020 年；Shoeybi 等，2019 年）。另一种做法是，Harlap 等（2018 年）和 Huang 等（2019 年）提出使用基于流水线的模型并行，其中不同的层被划分到各个设备上，微批次被流水线化 到不同的层。最后，乘积键网络（Lample 等，2019 年）通过基于给定层的输入标记表示进行可学习的嵌入查找，以扩大神经网络的容量。

我们的工作研究了在一种执行条件 计算的方法类别中的一个特定模型，在这种方法中，计算决策会根据输入动态做出。Cho 和 Bengio（2014）提出根据出现在模型隐藏状态中的某些位模式自适应选择权重。Eigen 等人（2013）构建了具有稠密矩阵乘法和 ReLU 激活的堆叠专家层，并在抖动 MNIST 和单调语音上展示了有前景的结果。在计算机视觉中，Puigcerver 等人（2020）在上游预训练期间基于语义类别手动路由词元，然后根据下游任务选择要使用的相关专家。

在现代深度学习架构的背景下，专家混合（MoE）在 Shazeer 等（2017）中被证明是有效的。该工作添加了一个 MoE 层，堆叠在 LSTM（Hochreiter 和 Schmidhuber, 1997）层之间，并将标记分别路由到专家的组合。这在语言建模和机器翻译基准上取得了最先进结果。随后，Mesh TensorFlow 库（Shazeer 等，2018）将 MoE 层重新引入到 Transformer 架构中，将 MoE 层作为 FFN 层的替代，然而，并没有配套的自然语言处理结果。更近些时候，随着机器学习基础设施的进步，GShard（Lepikhin 等，2020）扩展了 XLA 编译器，使用 MoE Transformer 显著提升了跨 100 种语言的机器翻译。最后，Fan 等（2021）选择了一种不同的确定性 MoE 策略，将模型参数划分为不重叠的语言组。

在 Transformer 注意力模式中，沿序列长度维度（ L ）的稀疏性是一种成功的技术，可将注意力复杂度从 $O(L^2)$ 降低（Child 等，2019；Correia 等，2019；Sukhbaatar 等，2019；Kitaev 等，2020；Zaheer 等，2020；Beltagy 等，2020）。这使得可以学习比以往更长的序列。本文采用的 Switch Transformer 版本未使用注意力稀疏性，但这些技术是互补的，作为未来工作，它们可以结合起来，有望改进需要长上下文的任务的学习。

7. 讨论

我们提出并讨论关于 Switch Transformer 以及一般稀疏专家模型的问题，这里的稀疏性指的是权重，而不是注意力模式。

Switch Transformer 是否仅因庞大的参数数量而表现更好？ 是的，而且这是有意为之！参数（独立于所使用的总 FLOPs）是扩展神经语言模型的一个有用维度。大规模模型已被充分证明表现更好（Kaplan 等，2020 年）。但在本例中，我们在使用相同计算资源的情况下，模型具有更高的样本效率并且更快。

我用不上超级计算机——这对我仍然有用吗？ 尽管这项工作主要聚焦于极其大型的模型，我们也发现，即便只有两个专家的模型，也能提升性能，同时轻松满足常见 GPU 或 TPU 的内存约束（详见附录 D）。因此，我们认为我们的技术在小规模环境中同样有用。

稀疏模型是否在速度-精度帕累托曲线上优于稠密模型？ 是的。在各种不同的模型规模下，稀疏模型在每步和墙钟时间上都优于稠密模型。我们的受控实验表明，在计算和时间固定的情况下，稀疏模型优于稠密模型。

我无法部署万亿参数模型——我们能缩小这些模型吗？ 我们无法完全保留模型质量，但通过将我们的稀疏模型蒸馏为稠密模型，可以实现 10 到 100 倍的压缩率，同时达到专家模型质量增益的 $\approx 30\%$ 。

为什么要使用 Switch Transformer 而不是模型并行的稠密模型？ 从时间角度来看，Switch Transformer 相比具有分片参数的稠密模型（图6）可以高效得多。此外，我们指出，这一决策并不相互排斥——我们

可以（而且确实）在 SwitchTransformers 中使用模型并行，从而提高每个标记的 FLOPs，但也会带来传统的模型并行的减速。

为什么稀疏模型尚未得到广泛使用？ 尝试稀疏模型的动机被在扩展稠密模型方面取得的巨大成功所抑制（正如 Hooker（2020）所论述的，这种成功部分由与深度学习硬件的协同适应所驱动）。此外，稀疏模型还面临多种问题，包括（1）模型复杂度，（2）训练困难，以及（3）通信开销。Switch Transformer 在缓解这些问题方面取得了进展。

8. 未来工作

本文提出了一个简化的架构、改进的训练流程，以及关于稀疏模型如何扩展的研究。然而，仍然存在许多开放的未来方向，我们在此简要描述：

1. 一个重要挑战是进一步提升规模最大的模型的训练稳定性。尽管我们的稳定性技术对 Switch-Base、Switch-Large 和 Switch-C 模型是有效的（未观察到不稳定性），但对 Switch-XXL 仍然不足。我们已经为稳定这些模型采取了早期的步数，我们认为其中一些方法对大规模模型具有普遍意义，包括使用用于提升稳定性的正则化项以及经过调整的梯度裁剪形式，但这一问题仍未解决。
2. 一般来说，我们发现预训练质量的提升会带来更好的下游结果（附录E），不过有时也会出现显著的反常情况。例如，尽管在对 C4 数据集建模时困惑度相近，参数量为 1.6T 的 Switch-C 在 SQuAD 上的精确匹配分数仅为 87.7，明显不如更小的 Switch-XXL 模型的 89.6。一个显著差异是，Switch-XXL 模型采用的每个标记的 FLOPs 是 Switch-C 模型的 ≈ 10 倍，尽管其唯一的参数少了 ≈ 4 倍（3950 亿 vs 1.6T）。这表明微调质量、*FLOPs per token* 和 *number of parameters* 之间存在一种尚未充分理解的依赖关系。
3. 开展对缩放关系的全面研究，以指导融合数据、模型和专家并行的架构设计。理想情况下，给定某种硬件配置的规格（计算、内存、通信），就可以更快速地设计出一个最优模型。反之，这也可能有助于未来硬件的设计。
4. 我们的工作属于自适应计算算法家族。我们的方法始终使用相同的同质专家，但未来的设计（在更灵活的基础设施的支持下）可以支持异质专家。这将使在需要更多计算时通过路由到更大的专家来实现更灵活的自适应——也许用于更难的样本。
5. 在 Transformer 的 FFN 层之外研究专家层。我们发现初步证据表明，这同样可以提升模型质量。在附录A中，我们报告了将其添加到自注意力层中带来的质量提升，其中我们

该层替换了生成 Q、K、V 的权重矩阵。然而，由于使用 bfloat16 格式时存在训练不稳定性，我们暂时将此留作未来工作。

6. 在新的模态和跨不同模态中检验 Switch Transformer。到目前为止，我们只考虑了语言，但我们相信，模型稀疏性同样能在新的模态以及多模态网络中带来优势。

这个清单很容易扩展，但我们希望这能让人了解我们正在思考的挑战类型，以及我们认为有前景的未来方向。

9. 结论

Switch Transformer 是可扩展且有效的自然语言学习器。我们将专家混合加以简化，提出了一种易于理解、训练稳定、且相比同等规模的稠密模型样本效率远高的架构。我们发现，这些模型在多样的自然语言任务和不同的训练范式中表现出色，包括预训练、微调和多任务训练。这些进展使得训练拥有从数万亿到万亿参数的模型成为可能，并且相较于稠密的 T5 基线取得了显著的加速。我们希望我们的工作能推动将稀疏模型作为一种有效的架构，并促使研究者和从业者在自然语言任务乃至更广的领域考虑采用这些灵活的模型。

致谢

作者谨在此感谢 Margaret Li，在数月内就算法改进提供了关键见解，并对实证研究提出了建议。还要感谢 Hugo Larochelle 提供睿智的建议并对初稿提出了澄清性意见，Irwan Bello 提供了详细意见并进行了细致修订，Colin Raffel 和 Adam Roberts 就神经网络模型与 T5 代码库提供了及时建议，Yoshua Bengio 就自适应计算研究给予了指导与鼓励，Jascha Sohl-dickstein 为稳定新的大规模模型和论文修订提出了有趣的新方向，以及 Google Brain 团队就本文进行了富有成效的讨论。最后感谢 Blake Hechtman 在性能剖析以及提升我们模型的训练性能方面提供了不可或缺的帮助。

A. 用于注意力的 Switch

Shazeer 等 (2018年)；Lepikhin 等 (2020年) 通过将 MoE 层添加到 Transformer 的稠密前馈网络 (FFN) 计算中，设计了 MoE Transformer (混合专家 Transformer)

(Shazeer 等, 2017 年)。类似地，我们的工作也在 Transformer 中替换了 FFN 层，但我们在此简要探索另一种设计。我们将 Switch 层添加到 Transformer 自注意力层中。为此，如图10所示，我们将生成查询、键和值的可训练权重矩阵替换为 Switch 层。

表10记录了若干变体在固定步数之后的质量以及训练时间。尽管我们发现了改进，但我们也发现这些层在使用 bfloat16 精度时更不稳定，因此没有将它们纳入最终变体。

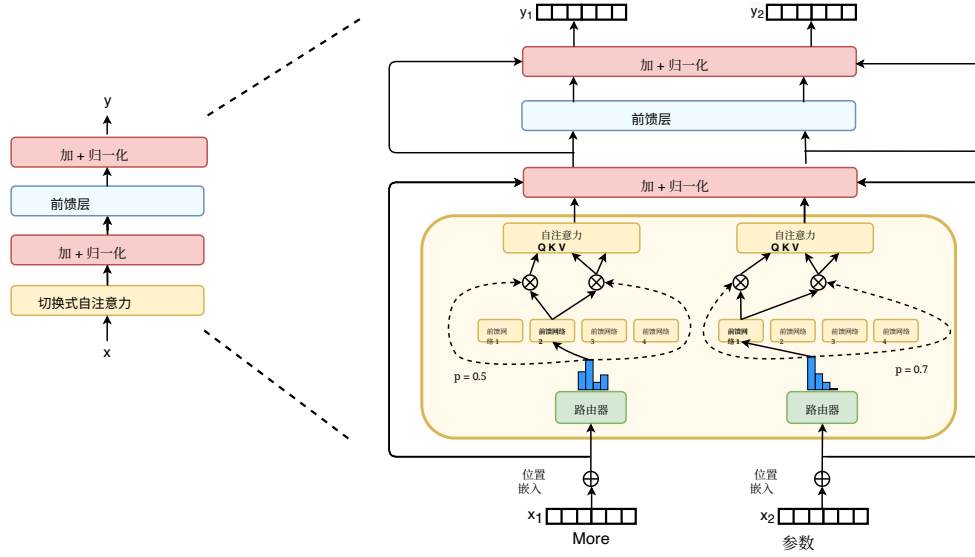


图10: 注意力中的 Switch 层。我们图示如何将 Switch 层融入自注意力 Transformer 块。对于每个标记（此处展示两个标记， x_1 = “更多”和 x_2 = “参数”），一组权重用于生成查询，另一组独立的权重用于生成共享的键和值。我们尝试让每个专家既可以是一个线性运算，也可以是一个 FFN，这在本文中始终如此。尽管我们发现这样做带来了质量提升，但与低精度数值格式一起使用时更不稳定，因此留待未来工作。

然而，当这些层确实能够稳定训练时，我们认为初步的积极结果表明了一个前景可期的未来方向。

模型	精度	质量 @100k 步 (↑)	质量 @16H (↑)	速度 (样本/秒) (↑)
专家 FF	float32	-1.548	-1.614	1480
专家注意力	float32	-1.524	-1.606	1330
专家注意力	bfloat16	[发散]	[发散]	—
专家 FF + 注意力	float32	-1.513	-1.607	1240
专家 FF + 注意力	bfloat16	[发散]	[发散]	—

表10: Switch 注意力层结果。所有模型都有 32 个专家，并以每个批次 524k 个标记进行训练。专家 FF 是指在 Transformer 中由专家替换 FFN，这是本文贯穿始终的标准设置。专家 FF + 注意力 是指由专家同时替换 FFN 和自注意力层。使用 bfloat16 精度进行训练时，具有专家注意力的模型会发散。

B. 使用 *No-Token-Left-Behind* 防止标记丢弃

由于 TPU 加速器上的软件限制，我们的张量形状必须是静态大小。因此，每个专家都有一个有限且固定的容量来处理标记表示。然而，这对我们的模型带来了问题：该模型在运行时动态路由标记，可能导致在专家之间的不均匀分布。如果发送到某个专家的标记数量少于专家容量，那么可以通过填充来完成计算——这虽然对硬件的使用效率不高，但在数学上是正确的。然而，当发送到某个专家的标记数量大于其容量（专家溢出）时，需要一种协议来处理。Lepikhin 等（2020）改造了一种专家混合模型，并通过残差连接在不处理的情况下将其表示传递到下一层以应对专家溢出，我们也采用这种做法。

我们怀疑，对标记不进行任何计算会非常浪费，尤其是因为如果某个专家发生了溢出，这意味着另一个专家会有额外的容量。基于这一直觉，我们提出了 *No-Token-Left-Behind*，它会对最初被路由到发生溢出的专家的任何标记进行迭代式重新路由。图11展示了该方法的示意图，这使我们能够保证在训练和推理过程中几乎不会有标记被丢弃。我们假设这可以提升性能并进一步稳定训练，但我们未发现实证收益。我们怀疑，一旦网络学会了不同标记与专家之间的关联，如果这种关联被改变（例如将某个标记发送给其第二高的专家），则性能可能会下降。

C. 鼓励跨专家的探索

在每个专家层，路由器决定将标记发送到哪个专家。这是在可用专家集合上的离散决策，条件取决于关于标记表示的信息。基于输入的标记表示，路由器确定最佳专家；然而，它没有关于选择其他专家会表现如何的反事实信息。与强化学习中类似，出现了经典的探索-利用困境（Sutton 与 Barto，2018）。这些问题也被 Rosenbaum 等（2017）以不同方式指出并加以解决，并在多任务学习中取得了成功。这种特定设置与上下文多臂赌博机（Robbins，1952）最为相符。确定性地选择顶级专家总是等同于一种利用型策略——我们考虑平衡探索以寻求更好的专家分配。

为引入探索，我们考虑了几种方法：1) 确定性或 argmax （取最大）2) 从 softmax 分布中采样 3) 在输入表示上施加输入 dropout 4) 在输入表示上施加乘性抖动噪声。所产生的对模型质量的影响见表11。在整篇工作中，我们使用输入抖动来注入噪声，因为我们发现它在经验上表现最好。

D. 低计算量条件下的 Switch Transformer

Switch Transformer 在小规模以及具有数千个核心和万亿参数的场景中同样是一种有效的架构。我们的许多先前的实验曾在

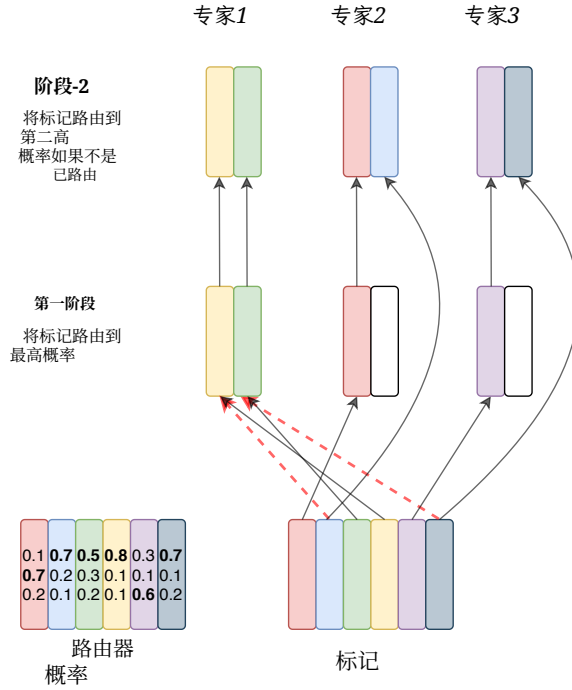


图11: *No-Token-Left-Behind Routing* 的示意图。阶段1等同于 *Switch* 路由，其中标记已路由至由路由器给出最高概率的专家。在阶段2，我们查看所有已溢出的标记，并将它们路由到具有第二高概率的专家。如果第二高概率的专家拥有过多标记，标记仍可能发生溢出，但这使得大多数标记都能被路由。该过程可以迭代，以保证几乎不会丢弃任何标记。

模型	质量 (负对数困惑度) ($\uparrow$)
argmax (取最大)	-1.471
Softmax 采样	-1.570
输入 dropout	-1.480
输入抖动	-1.468

表11: 路由器探索策略。基于负对数困惑度度量的 *Switch Transformer* 在采用不同选择专家的随机策略下的质量（数值越低越好）。各变体之间在速度性能上没有实质性差异。

10B+parameter 模型的规模下进行，但我们在图12中表明，即使只有 2 位专家，也能相比 FLOP 匹配的对照模型取得显著提升。即便无法轻易获得超级计算机，使用 2、4 或 8 位专家来训练 *Switch Transformer*（我们通常建议每个核心对应一位专家）也能较 T5 稠密基线带来稳健的提升。

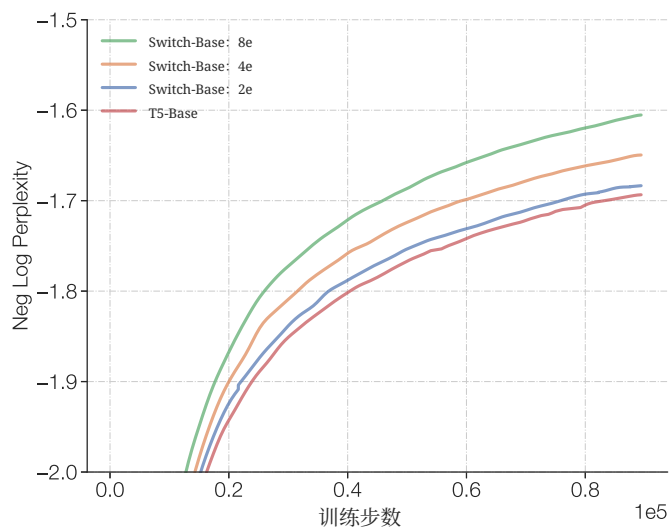


图12: 少量专家的 Switch Transformer。即使在专家非常少的情况下, Switch Transformer 也优于基线。这里我们展示在非常小的规模下的扩展性特性, 在使用 2、4 和 8 位专家时, 我们优于 T5-Base 模型。

E. 上游到下游模型性能的关系

模型在预训练目标上的质量无法保证能转化为下游任务的结果。图13展示了稠密和 Switch 模型在 C4 预训练任务上的上游模型质量与两项下游任务指标之间的相关性：平均 SuperGLUE 性能和 TriviaQA 分数。我们选择这两个任务，因为一个考察模型的推理，另一个考察其事实知识。

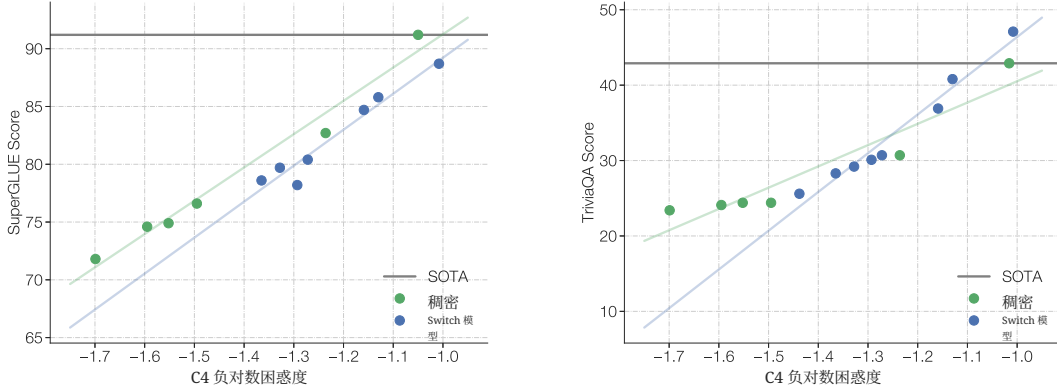


图13：上游预训练质量与下游模型质量。我们在 SuperGLUE 和 TriviaQA 上将上游性能与下游质量相关联（SOTA 记录不包含 SSM），它们分别是推理和知识密集型基准（验证集）。我们发现，与基线一样，Switch 模型也会随上游预训练任务的改进而提升。对于 SuperGLUE，我们发现负对数困惑度与平均 SuperGLUE 分数之间呈大致线性关系。然而，在固定困惑度下，尤其在大规模范围内，稠密模型往往表现更好。相反，在知识密集型任务 TriviaQA 上，我们发现 Switch Transformer 可能遵循更优的缩放关系——在给定的上游困惑度下，它优于稠密对应模型。还需要更多统计数据（收集代价高，留待未来工作）来确认这些观察结果。

我们发现存在一致的相关性，这表明对于基线和 Switch 模型，改进的预训练会带来更好的下游结果。此外，在固定的上游困惑度下，我们发现在小到中等模型规模范围内，Switch 模型和稠密模型的表现相近。然而，在最大模型规模范围（T5-11B/T5-XXL）中，我们最大的 Switch 模型（如第5.6节所述）并不总是能将其上游困惑度很好地转化为在 SuperGLUE 任务上的下游微调表现。这需要在未来进一步研究，以充分释放稀疏模型的潜力。理解使用专家模型进行微调时的动态非常复杂，并且取决于正则化、负载均衡以及微调超参数。

F. Pseudo Code for Switch Transformers

Pseudocode for Switch Transformers in Mesh Tensorflow (Shazeer et al., 2018). No model parallelism is being used for the below code (see 5.4 for more details).

```
import mesh_tensorflow as mtf

def load_balance_loss(router_probs, expert_mask):
    """Calculate load-balancing loss to ensure diverse expert routing."""
    # router_probs is the probability assigned for each expert per token.
    # router_probs shape: [num_cores, tokens_per_core, num_experts]
    # expert_index contains the expert with the highest router probability in one-hot format.
    # expert_mask shape: [num_cores, tokens_per_core, num_experts]

    # For each core, get the fraction of tokens routed to each expert.
    # density_1 shape: [num_cores, num_experts]
    density_1 = mtf.reduce_mean(expert_mask, reduced_dim=tokens_per_core)

    # For each core, get fraction of probability mass assigned to each expert
    # from the router across all tokens.
    # density_1.proxy shape: [num_cores, num_experts]
    density_1.proxy = mtf.reduce_mean(router_probs, reduced_dim=tokens_per_core)

    # density_1 for a single core: vector of length num_experts that sums to 1.
    # density_1.proxy for a single core: vector of length num_experts that sums to 1.
    # Want both vectors to have uniform allocation (1/num_experts) across all num_expert elements.
    # The two vectors will be pushed towards uniform allocation when the dot product is minimized.
    loss = mtf.reduce_mean(density_1.proxy * density_1) * (num_experts ^ 2)
    return loss
```

Figure 14: Pseudo code for the load balance loss for Switch Transformers in Mesh Tensorflow.

```

import mesh_tensorflow as mtf

def router(inputs, capacity_factor):
    """Produce the combine and dispatch tensors used for sending and
    receiving tokens from their highest probability expert. """
    # Core layout is split across num_cores for all tensors and operations.
    # inputs shape: [num_cores, tokens_per_core, d_model]

    router_weights = mtf.Variable(shape=[d_model, num_experts])

    # router_logits shape: [num_cores, tokens_per_core, num_experts]
    router_logits = mtf.einsum([inputs, router_weights], reduced_dim=d_model)

    if is_training:
        # Add noise for exploration across experts.
        router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

    # Convert input to softmax operation from bfloat16 to float32 for stability.
    router_logits = mtf.to_float32(router_logits)

    # Probabilities for each token of what expert it should be sent to.
    router_probs = mtf.softmax(router_logits, axis=-1)

    # Get the top-1 expert for each token. expert_gate is the top-1 probability
    # from the router for each token. expert_index is what expert each token
    # is going to be routed to.
    # expert_gate shape: [num_cores, tokens_per_core]
    # expert_index shape: [num_cores, tokens_per_core]
    expert_gate, expert_index = mtf.top_1(router_probs, reduced_dim=num_experts)

    # expert_mask shape: [num_cores, tokens_per_core, num_experts]
    expert_mask = mtf.one_hot(expert_index, dimension=num_experts)

    # Compute load balancing loss.
    aux_loss = load_balance_loss(router_probs, expert_mask)

    # Experts have a fixed capacity, ensure we do not exceed it. Construct
    # the batch indices, to each expert, with position_in_expert
    # make sure that not more that expert_capacity examples can be routed to
    # each expert.
    position_in_expert = mtf.cumsum(expert_mask, dimension=tokens_per_core) * expert_mask

    # Keep only tokens that fit within expert_capacity.
    expert_mask *= mtf.less(position_in_expert, expert_capacity)
    expert_mask_flat = mtf.reduce_sum(expert_mask, reduced_dim=experts_dim)

    # Mask out the experts that have overflowed the expert capacity.
    expert_gate *= expert_mask_flat

    # combine_tensor used for combining expert outputs and scaling with router probability.
    # combine_tensor shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    combine_tensor = (
        expert_gate * expert_mask_flat *
        mtf.one_hot(expert_index, dimension=num_experts) *
        mtf.one_hot(position_in_expert, dimension=expert_capacity))

    # Cast back outputs to bfloat16 for the rest of the layer.
    combine_tensor = mtf.to_bfloat16(combine_tensor)

    # Create binary dispatch tensor that is 1 if the token gets routed to the corresponding expert.
    # dispatch_tensor shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    dispatch_tensor = mtf.cast(combine_tensor, tf.bool)

    return dispatch_tensor, combine_tensor, aux_loss

```

Figure 15: Pseudo code for the router for Switch Transformers in Mesh Tensorflow.

```

import mesh_tensorflow as mtf

def switch_layer(inputs, n, capacity_factor, num_experts):
    """Distributed switch transformer feed-forward layer."""
    # num_cores (n) = total cores for training the model (scalar).
    # d_model = model hidden size (scalar).
    # num_experts = total number of experts.
    # capacity_factor = extra buffer for each expert.
    # inputs shape: [batch, seq_len, d_model]
    batch, seq_len, d_model = inputs.get_shape()

    # Each core will route tokens_per_core tokens to the correct experts.
    tokens_per_core = batch * seq_len / num_cores

    # Each expert will have shape [num_cores, expert_capacity, d_model].
    # Each core is responsible for sending expert_capacity tokens
    # to each expert.
    expert_capacity = tokens_per_core * capacity_factor / num_experts

    # Reshape to setup per core expert dispatching.
    # shape: [batch, seq_len, d_model] -> [num_cores, tokens_per_core, d_model]
    # Core layout: [n, 1, 1] -> [n, 1, 1]
    inputs = mtf.reshape(inputs, [num_cores, tokens_per_core, d_model])

    # Core Layout: [n, 1, 1] -> [n, 1, 1, 1], [n, 1, 1, 1]
    # dispatch_tensor (boolean) shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    # dispatch_tensor is used for routing tokens to the correct expert.
    # combine_tensor (float) shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    # combine_tensor used for combining expert outputs and scaling with router
    # probability.
    dispatch_tensor, combine_tensor, aux_loss = router(inputs, expert_capacity)

    # Matmul with large boolean tensor to assign tokens to the correct expert.
    # Core Layout: [n, 1, 1], -> [1, n, 1, 1]
    # expert_inputs shape: [num_experts, num_cores, expert_capacity, d_model]
    expert_inputs = mtf.einsum([inputs, dispatch_tensor], reduce_dims=[tokens_per_core])

    # All-to-All communication. Cores split across num_cores and now we want to split
    # across num_experts. This sends tokens, routed locally, to the correct expert now
    # split across different cores.
    # Core layout: [1, n, 1, 1] -> [n, 1, 1, 1]
    expert_inputs = mtf.reshape(expert_inputs, [num_experts, num_cores, expert_capacity, d_model])

    # Standard feed forward computation, where each expert will have its own
    # unique set of parameters.
    # Total unique parameters created: num_experts * (d_model * d_ff * 2).
    # expert_outputs shape: [num_experts, num_cores, expert_capacity, d_model]
    expert_outputs = feed_forward(expert_inputs)

    # All-to-All communication. Cores are currently split across the experts
    # dimension, which needs to be switched back to being split across num_cores.
    # Core Layout: [n, 1, 1, 1] -> [1, n, 1, 1]
    expert_outputs = mtf.reshape(expert_outputs, [num_experts, num_cores, expert_capacity, d_model])

    # Convert back to input shape and multiply outputs of experts by the routing probability.
    # expert_outputs shape: [num_experts, num_cores, tokens_per_core, d_model]
    # expert_outputs.combined shape: [num_cores, tokens_per_core, d_model]
    # Core Layout: [1, n, 1, 1] -> [n, 1, 1]
    expert_outputs_combined = mtf.einsum([expert_outputs, combine_tensor], reduce_dims=[tokens_per_core])

    # Remove tokens_per_core shapes used for local routing dispatching to match input shape.
    # Core Layout: [n, 1, 1] -> [n, 1, 1]
    outputs = mtf.reshape(expert_outputs_combined, [batch, seq_len, d_model])
    return outputs, aux_loss
    
```

Figure 16: Pseudo code of the Switch Transformer layer in Mesh Tensorflow.

References

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*, pages 265–283, 2016.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on free-base from question-answer pairs. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1533–1544, 2013.
- Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Kyunghyun Cho and Yoshua Bengio. Exponentially increasing the capacity-to-computation ratio for conditional computation in deep learning. *arXiv preprint arXiv:1406.7362*, 2014.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Gonçalo M Correia, Vlad Niculae, and André FT Martins. Adaptively sparse transformers. *arXiv preprint arXiv:1909.00015*, 2019.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- David Eigen, Marc’Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts. *arXiv preprint arXiv:1312.4314*, 2013.
- Angela Fan, Shruti Bhosale, Holger Schwenk, Zhiyi Ma, Ahmed El-Kishky, Siddharth Goyal, Mandeep Baines, Onur Celebi, Guillaume Wenzek, Vishrav Chaudhary, et al. Beyond english-centric multilingual machine translation. *Journal of Machine Learning Research*, 22(107):1–48, 2021.
- William Fedus, Ian Goodfellow, and Andrew M Dai. Maskgan: Better text generation via filling in the_. *arXiv preprint arXiv:1801.07736*, 2018.
- Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. Sparse gpu kernels for deep learning. *arXiv preprint arXiv:2006.10901*, 2020.
- Scott Gray, Alec Radford, and Diederik P Kingma. Gpu kernels for block-sparse weights. <https://openai.com/blog/block-sparse-gpu-kernels/>, 2017.

- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. *arXiv preprint arXiv:2002.08909*, 2020.
- Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.
- Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28, pages 1693–1701. Curran Associates, Inc., 2015. URL <https://proceedings.neurips.cc/paper/2015/file/afdec7005cc9f14302cd0474fd0f3c96-Paper.pdf>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Sara Hooker. The hardware lottery. *arXiv preprint arXiv:2009.06489*, 2020.
- Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyounJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in neural information processing systems*, pages 103–112, 2019.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6(2):181–214, 1994.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

- Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, pages 8548–8559, 2019.
- Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499*, 2021.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.
- Shashi Narayan, Shay B Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*, 2018.
- Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. Adversarial nli: A new benchmark for natural language understanding. *arXiv preprint arXiv:1910.14599*, 2019.
- Joan Puigcerver, Carlos Riquelme, Basil Mustafa, Cedric Renggli, André Susano Pinto, Sylvain Gelly, Daniel Keysers, and Neil Houlsby. Scalable transfer learning with expert models. *arXiv preprint arXiv:2009.13239*, 2020.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683*, 2019.
- Samyung Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimization towards training a trillion parameter models. *arXiv preprint arXiv:1910.02054*, 2019.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.
- Prajit Ramachandran and Quoc V Le. Diversity and depth in per-example routing models. In *International Conference on Learning Representations*, 2018.
- Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

- Adam Roberts, Colin Raffel, and Noam Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.
- Clemens Rosenbaum, Tim Klinger, and Matthew Riemer. Routing networks: Adaptive selection of non-linear functions for multi-task learning. *arXiv preprint arXiv:1711.01239*, 2017.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8732–8740, 2020.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, 2019.
- Noam Shazeer. Glu variants improve transformer, 2020.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyukJoong Lee, Mingsheng Hong, Cliff Young, et al. Mesh-tensorflow: Deep learning for supercomputers. In *Advances in Neural Information Processing Systems*, pages 10414–10423, 2018.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using gpu model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014. URL <http://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf>.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.
- Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. *arXiv preprint arXiv:1905.07799*, 2019.
- Rich Sutton. The Bitter Lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. Stanford University, 2018.
- Wilson L Taylor. “cloze procedure”: A new tool for measuring readability. *Journalism quarterly*, 30(4):415–433, 1953.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, pages 3266–3280, 2019.
- Shibo Wang and Pankaj Kanwar. Bfloat16: The secret to high performance on cloud tpus. *Google Cloud Blog*, 2019.
- Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. mt5: A massively multilingual pre-trained text-to-text transformer. *arXiv preprint arXiv:2010.11934*, 2020.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding, 2020.
- Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *arXiv preprint arXiv:2007.14062*, 2020.